

Les structures de données Spark

Module 3 — RDD, DataFrame, Dataset et Spark SQL

Objectifs pédagogiques

- Comprendre le concept de RDD et ses propriétés fondamentales
- Créer et manipuler des DataFrames et des RDDs
- Distinguer les cas d'usage de chaque structure de données
- Utiliser Spark SQL pour interroger des données structurées
- Lire et interpréter un plan d'exécution Spark (DAG)

1. Vue d'ensemble

3 niveaux d'abstraction

Du plus bas niveau au plus haut niveau — plus le niveau est élevé, plus Catalyst peut optimiser

Avantages d'un niveau élevé :



Dataset[T]

API Scala/Java fortement typée

→ API plus expressive et concise

DataFrame

= Dataset[Row] — API PySpark

→ Catalyst peut davantage optimiser

RDD[T]

Resilient Distributed Dataset — bas niveau

→ Meilleures performances en général

En PySpark : on travaille principalement avec les DataFrames. Les RDD restent utiles pour les données non structurées ou les algorithmes de bas niveau. Le Dataset est réservé à Scala/Java.

2. Les RDD

Resilient Distributed Dataset

RDD — Définition et 5 propriétés fondamentales

Structure de données primitive de Spark — toutes les structures haut niveau reposent sur le RDD

Resilient

Tolérant aux pannes

Partition perdue → recalculée depuis le lignage (lineage)

Distributed

Distribué

Données découpées en partitions réparties sur le cluster

Dataset

Jeu de données

Collection d'éléments de n'importe quel type Python

Immuable

Immutable

Pas de modification en place — chaque transformation crée un nouveau RDD

Lazy evaluation

Évaluation paresseuse

Transformations planifiées, non exécutées — jusqu'à une action

Transformations vs Actions

Deux types d'opérations — seules les actions déclenchent l'exécution du DAG

Transformations (lazy)

Retournent un nouveau RDD — rien n'est calculé

```
rdd2 = rdd1.map(lambda x: x*2)
rdd3 = rdd2.filter(lambda x: x>4)
rdd4 = rdd3.flatMap(lambda x: [x,x+1])
# map, filter, flatMap, union,
# groupByKey, reduceByKey,
# sortBy, distinct, join...
```

Actions (eager)

Déclenchent l'exécution du DAG complet

```
rdd3.count()      # nb éléments
rdd3.collect()    # tout au Driver
rdd3.take(5)      # 5 premiers
rdd3.first()      # 1er élément
rdd3.reduce(lambda a,b: a+b)
rdd3.saveAsTextFile("output/")
```

⚠ **Règle d'or :** ne jamais appeler `.collect()` sur un RDD volumineux — toutes les données sont rapatriées en mémoire du Driver.

Création d'un RDD — 4 méthodes

```
sc = spark.sparkContext
```

```
# Méthode 1 : paralléliser une collection Python
```

```
rdd1 = sc.parallelize([1,2,3,4,5,6,7,8])
```

```
rdd1_p = sc.parallelize([1,2,3,4,5,6,7,8], numSlices=4)
```

```
# Méthode 2 : lire un fichier texte (1 ligne = 1 élément)
```

```
rdd_texte = sc.textFile("data/roman.txt")
```

```
rdd_glob = sc.textFile("data/*.txt") # plusieurs fichiers
```

```
# Méthode 3 : RDD Pair (clé-valeur) — accès à reduceByKey, groupByKey...
```

```
rdd_pair = sc.parallelize([("Paris", 1500.0), ("Lyon", 800.0), ("Paris", 2200.0)])
```

```
# Méthode 4 : depuis un DataFrame (conversion)
```

```
rdd_df = df.rdd # chaque élément est un objet Row
```

```
rdd_tp = df.rdd.map(tuple) # convertir en tuples Python
```


3. Les Dataframes

DataFrame — Structure de données de haut niveau

Collection distribuée de colonnes nommées et typées — structure recommandée par défaut

Pandas DataFrame

Même API conceptuelle, mais exécutée sur un cluster distribué — pas limité à la RAM d'une machine

Table SQL

Colonnes nommées et typées, requêtes via Spark SQL — analogie directe avec une table relationnelle

Fichier tabulaire

Analogue à un CSV ou Excel avec schéma — lecture/écriture native depuis CSV, JSON, Parquet, etc.

Avantage clé : le schéma est connu à l'avance → l'optimiseur **Catalyst** peut appliquer des optimisations automatiques (predicate pushdown, fusion de filtres, projection pushdown)

```
# Structure d'un DataFrame Spark
```

```
# +-----+-----+-----+-----+
```

```
# | nom   | ville | chiffre_affaires|
```

```
# | Alice | Paris |      1500.0 |
```

```
# | Bob   | Lyon  |       800.0 |
```

```
# +-----+-----+-----+-----+ # schéma : nom=String, ville=String, ca=Double
```

Création d'un DataFrame — 5 méthodes

Méthode 1 : depuis une liste Python (noms de colonnes)

```
data = [("Alice","Paris",1500.0), ("Bob","Lyon",800.0)]
```

```
df1 = spark.createDataFrame(data, schema=["nom","ville","ca"])
```

Méthode 2 : schéma explicite avec StructType (recommandé en production)

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType
```

```
schema = StructType([StructField("nom", StringType(), nullable=False),
```

```
    StructField(
```

```
    "ca", DoubleType(), nullable=True)])
```

Méthode 3 : lecture de fichiers

```
df_csv = spark.read.option("header","true").option("inferSchema","true").csv("ventes.csv")
```

```
df_json = spark.read.json("utilisateurs.json")
```

```
df_parquet = spark.read.parquet("data/transactions/") # format colonnaire — recommandé en production
```

Méthode 4 : depuis un RDD | Méthode 5 : depuis Pandas

```
df = rdd.toDF(["nom","age"]) # rdd → df
```

```
df = spark.createDataFrame(pdf) # pandas → spark (prototypage)
```


Schéma et types de données Spark

Type Spark	Type Python	Description
StringType	str	Chaîne de caractères
IntegerType	int	Entier 32 bits
LongType	int	Entier 64 bits
FloatType / DoubleType	float	Flottant 32 ou 64 bits
BooleanType	bool	Booléen
DateType	datetime.date	Date (sans heure)
TimestampType	datetime.datetime	Date + heure
ArrayType	list	Tableau d'éléments typés
MapType	dict	Dictionnaire clé-valeur
StructType	—	Structure imbriquée (objet JSON)

Bonne pratique : toujours définir un schéma **explicite** en production. L'option **inferSchema=True** est pratique mais lit deux fois les données et peut inférer des types incorrects.

Exploration et accès aux colonnes

```
# Aperçu des données
```

```
df.show()
```

```
df.show(5, truncate=False)
```

```
# Statistiques descriptives
```

```
df.describe().show() # mean, min, max
```

```
df.summary().show() # + percentiles
```

```
# Dimensions
```

```
df.count() # nb lignes
```

```
len(df.columns) # nb colonnes
```

```
df.printSchema() # types
```

```
from pyspark.sql import functions as F
```

```
# 3 syntaxes équivalentes pour select
```

```
df.select("nom", "ville")
```

```
df.select(df["nom"], df["ville"])
```

```
df.select(F.col("nom"), F.col("ville")) ← recommandé
```

```
# Valeurs nulles et distinctes
```

```
df.select("ville").distinct().show()
```

```
df.filter(F.col("ca").isNull()).count()
```

F.col() est la syntaxe recommandée pour accéder aux colonnes — pas d'ambiguïté avec les variables Python contrairement à **df.nom** (déconseillé si le nom est un mot réservé Python).

- df.dtypes → liste de (nom_colonne, type_string)
- df.schema → objet StructType complet

4. Le Dataset — notion complémentaire

API fortement typée — Scala/Java uniquement

En PySpark : un DataFrame est un Dataset[Row]

Spark SQL

Vues temporaires

Spark SQL — SQL standard sur les DataFrames

Particulièrement utile pour les Data Analysts et les transformations complexes en SQL

```
# Enregistrer comme vue temporaire
df.createOrReplaceTempView("ventes")

result = spark.sql("""
    SELECT ville,
           SUM(chiffre_affaires) AS ca_total,
           COUNT(*) AS nb_vendeurs
    FROM   ventes
    WHERE  chiffre_affaires > 500
    GROUP BY ville
    ORDER BY ca_total DESC
""")
result.show()
```

Équivalences SQL / API DataFrame

SELECT col1, col2	.select('col1','col2')
WHERE col > 100	.filter(F.col('col')>100)
GROUP BY col	.groupBy('col')
ORDER BY col DESC	.orderBy(F.col('col').desc())
JOIN ... ON	.join(df2,on='id',how='inner')
LIMIT 10	.limit(10)
DISTINCT	.distinct()

Vue globale : df.createOrReplaceGlobalTempView("ventes_global") — accessible depuis toutes les sessions via le préfixe `global_temp`.

6. RDD vs. Dataframe

RDD vs DataFrame — Tableau de décision

Critère	RDD	DataFrame ✓ (par défaut)
Performance	Moins optimisé (pas de Catalyst)	Optimisé automatiquement par Catalyst
API	Fonctionnelle : map, filter, reduce	SQL-like : select, filter, groupBy
Typage	Flexible — n'importe quel objet Python	Schéma strict (colonnes typées)
Lisibilité	Moins lisible pour les pipelines complexes	Plus lisible, plus déclaratif
Données non structurées	Idéal (texte brut, objets Python libres)	Nécessite un schéma
ML moderne	API legacy (MLlib RDD-based)	API moderne : MLlib DataFrame-based
Structured Streaming	Non supporté	Support natif

Règle pratique : utiliser le **DataFrame par défaut** (90% des pipelines PySpark). Recourir au RDD uniquement pour les données non structurées, le contrôle fin du partitionnement, ou les algorithmes Python complexes difficiles à vectoriser.

7. Les DAG et le plan d'exécution

Le Catalyst Optimizer — 3 optimisations clés

Catalyst analyse le DAG logique et réécrit automatiquement le plan avant l'exécution

Predicate pushdown

*Les filtres sont appliqués le plus tôt possible
— idéalement à la lecture du fichier*

```
df1.join(df2,'id').filter(F.col('montant')>100)
```

→ Catalyst : `df1.filter(...).join(df2,'id')`

Projection pushdown

*Seules les colonnes nécessaires sont lues
depuis la source — crucial pour Parquet*

```
df.select('nom','montant').filter(...)
```

→ Spark ne lit QUE 'nom' et 'montant' dans le fichier

Fusion de filtres

*Plusieurs filtres consécutifs sont fusionnés en
un seul passage sur les données*

```
df.filter(A).filter(B).filter(C)
```

→ Catalyst : `df.filter(A AND B AND C)`

Lire un plan d'exécution — explain()

De bas en haut : FileScan → Filter → HashAggregate → Exchange (shuffle) → HashAggregate final

```
# Plan physique simple
df_res.explain()

# Plan détaillé — 4 niveaux :
df_res.explain(mode="extended")

# Parsed / Analyzed / Optimized / Physical
df_res.explain(mode="formatted")
```

== Physical Plan ==

```
AdaptiveSparkPlan  ← AQE dynamique
*(2) HashAggregate(keys=[region])
+- Exchange hashpartitioning ← SHUFFLE
   +- *(1) HashAggregate  ← partiel
      +- *(1) Filter(montant>100)
         +- FileScan parquet ← source
```

Jobs, Stages et Tasks

ACTION (.show(), .count()...)

└─ JOB (1 par action)

└─ STAGE 1 (calculs sans shuffle — tasks parallèles sur chaque partition)

└─ STAGE 2 (après l'Exchange/shuffle — nouveau découpage des données)

Règle : un nouveau Stage est créé à chaque opération de shuffle (groupBy, join, repartition, sortBy...)

8. Conversions entre structures

Conversions entre structures de données

```
# — DataFrame → RDD —  
rdd = df.rdd # RDD de Row objects  
rdd_tuples = df.rdd.map(tuple) # RDD de tuples Python  
# — RDD → DataFrame —  
df = rdd.toDF(["col1","col2"]) # noms de colonnes  
df = spark.createDataFrame(rdd, schema=schema)  
# — DataFrame → Pandas —  
pdf = df.toPandas() # ⚠ Rapatrie TOUT en mémoire Driver  
pdf = df.limit(1000).toPandas() # ✅ Toujours limiter !  
# — Pandas → DataFrame —  
df = spark.createDataFrame(pdf)  
# — DataFrame → Spark SQL —  
df.createOrReplaceTempView("ma_table")  
result = spark.sql("SELECT * FROM ma_table")
```

Performance : les conversions RDD ↔ DataFrame ont un coût (sérialisation). Préférer rester dans le même paradigme. La conversion Pandas → Spark en prototypage est acceptable ; Spark → Pandas en production est dangereuse sur gros volumes.

Résumé du module

Concept	Points clés à retenir
RDD	Structure primitive — immuable, distribuée, tolérante aux pannes via le lignage
Lazy evaluation	Aucun calcul avant une action — Spark optimise d'abord, calcule ensuite
DataFrame	Collection de colonnes typées avec schéma — structure recommandée par défaut
Schéma	Connu à l'avance → permet l'optimisation Catalyst et la validation des types
Spark SQL	SQL standard sur des DataFrames via des vues temporaires
Catalyst	Optimiseur : predicate pushdown, projection pushdown, fusion de filtres
DAG	Graphe des transformations — découpé en Jobs → Stages → Tasks à l'exécution
RDD vs DataFrame	DataFrame par défaut (90% des cas) — RDD pour le non structuré ou le bas niveau

Exercices

Ex. 1 — Word Count *(30 min)* Implémenter le comptage de mots sur un fichier texte via RDD : `textFile` → `flatMap` → `map` → `reduceByKey` → `sortBy` → `take(20)`.

Ex. 2 — Schéma et types *(20 min)* Créer un DataFrame de films (titre, annee, genre, note, duree_min) avec StructType explicite — filtrer >120min et note>7.5, calculer la note moyenne par genre.

Ex. 3 — SQL vs API DataFrame *(25 min)* Requête SQL : meilleur film par genre. Écrire l'équivalent en API DataFrame. Comparer les plans d'exécution avec `.explain()` — sont-ils identiques ?

Ex. 4 — Exploration du DAG *(25 min)* Pipeline `filter` → `withColumn` → `groupBy` → `agg` → `orderBy`. Avant `.show()`, prédire le nombre de jobs. Compter les Exchange dans `.explain()`, vérifier les stages dans la Spark UI.

Pour aller plus loin

Documentation

- Documentation RDD : spark.apache.org/docs/latest/rdd-programming-guide.html
- Documentation DataFrame / Spark SQL : spark.apache.org/docs/latest/sql-programming-guide.html
- Fonctions Spark SQL natives : spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions.html

Articles fondateurs

Zaharia et al. — *Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing*

NSDI 2012 — fondements des RDD

Armbrust et al. — *Spark SQL: Relational Data Processing in Spark*

SIGMOD 2015 — fondements du Catalyst Optimizer et des DataFrames

Module suivant → Module 4 : Transformations de type mapper — map, flatMap, withColumn, UDFs et Pandas UDFs